Technical Report

FOOD DELIVERY BACKEND

Sara AIT ALI

Introduction

The goal of this project was to design and implement a cloud-native backend system for a food delivery platform. The system needs to manage restaurants, menus, and customer orders while ensuring scalability, service isolation, and persistent storage.

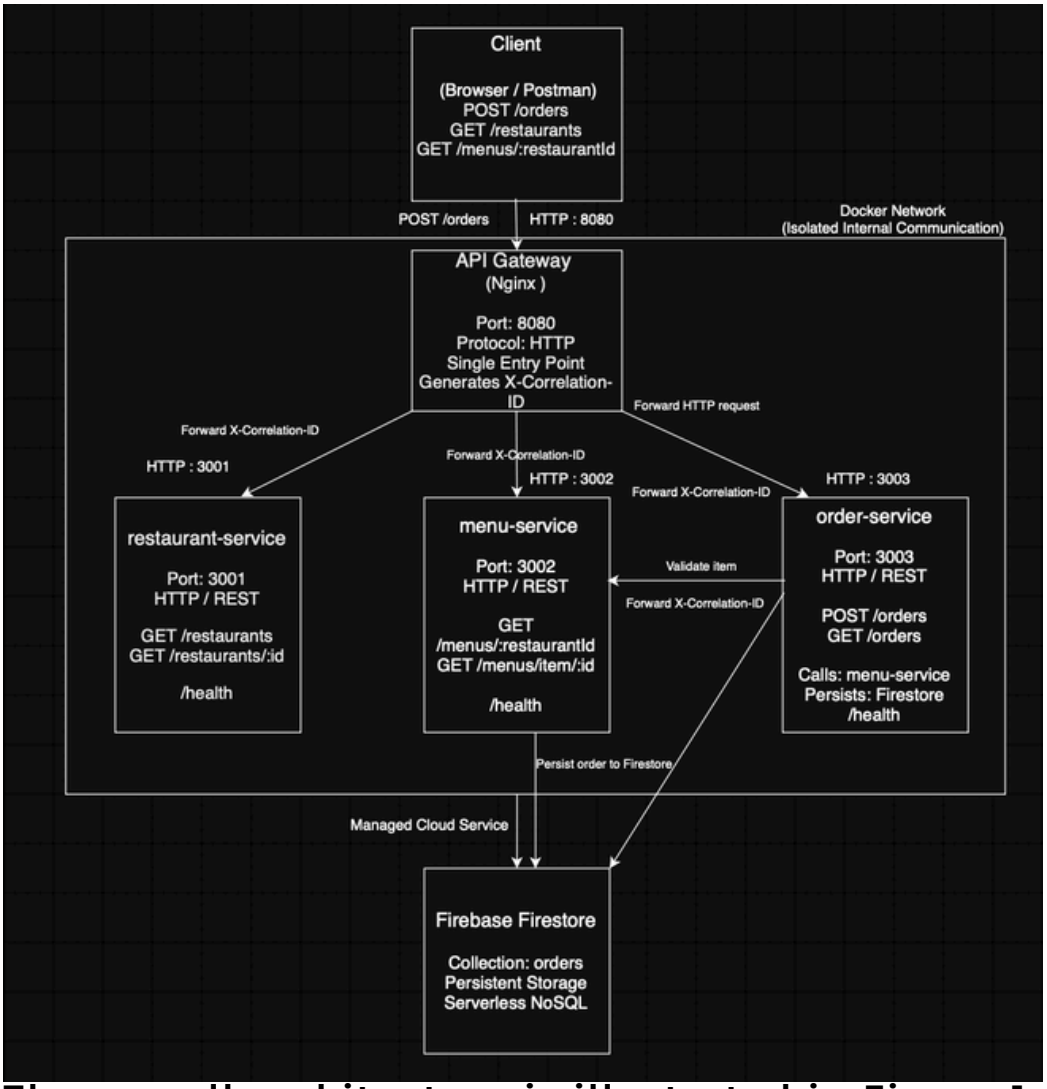
Instead of building the system as a single monolithic application, I chose a microservices architecture. This approach allows each domain (restaurants, menus, orders) to be handled independently. It improves modularity, makes debugging easier, and reflects how modern backend systems are structured in production environments.

Another objective was to apply cloud-native principles such as containerization, service isolation, environment-based configuration, and integration with a managed cloud service. For this reason, Docker was used for orchestration and Firebase Firestore was selected as the persistent storage solution.

This report explains the main architectural decisions made during the implementation of the system and provides a technical justification for those choices.

System Architecture Overview

The system is composed of four main components: the API Gateway, three microservices (restaurant-service, menu-service, and order-service), and Firebase Firestore as a managed cloud database.
The API Gateway (Nginx) acts as the single entry point to the system and listens on port 8080. All client requests must go through this gateway.
Internal service ports (3001, 3002, 3003) are not exposed outside the Docker network.
Each service has a clear responsibility:
The restaurant-service manages restaurant data.
The menu-service manages menu items for each restaurant.
The order-service handles order creation and retrieval.
The order-service is also responsible for persisting order data in Firebase Firestore.
All services communicate using REST over HTTP inside an isolated Docker network. When a client sends a request to create an order, the API Gateway forwards the request to the order-service. The order-service then calls the menu-service to validate the selected item before saving the order to Firestore.
This separation of concerns keeps the system modular and easier to maintain.



The overall architecture is illustrated in Figure 1.

API Gateway and Routing Design

The API Gateway plays an important role in the system architecture. Instead of exposing each microservice directly, all external traffic is routed through Nginx, which listens on port 8080.

This design ensures that internal service ports (3001, 3002, 3003) remain private inside the Docker network. Clients cannot access services directly, which improves security and keeps the system structure clean.

The gateway is responsible for routing requests based on URL paths:

Requests starting with /restaurants are forwarded to the restaurant-service.

Requests starting with /menus are forwarded to the menu-service.

Requests starting with /orders are forwarded to the order-service.

In addition to routing, the API Gateway generates an X-Correlation-ID for each incoming request. This ID is forwarded to all internal services and used for request tracing across the system.

Using a gateway simplifies the architecture and makes it easier to extend the system in the future, for example by adding authentication or rate limiting in one central place.

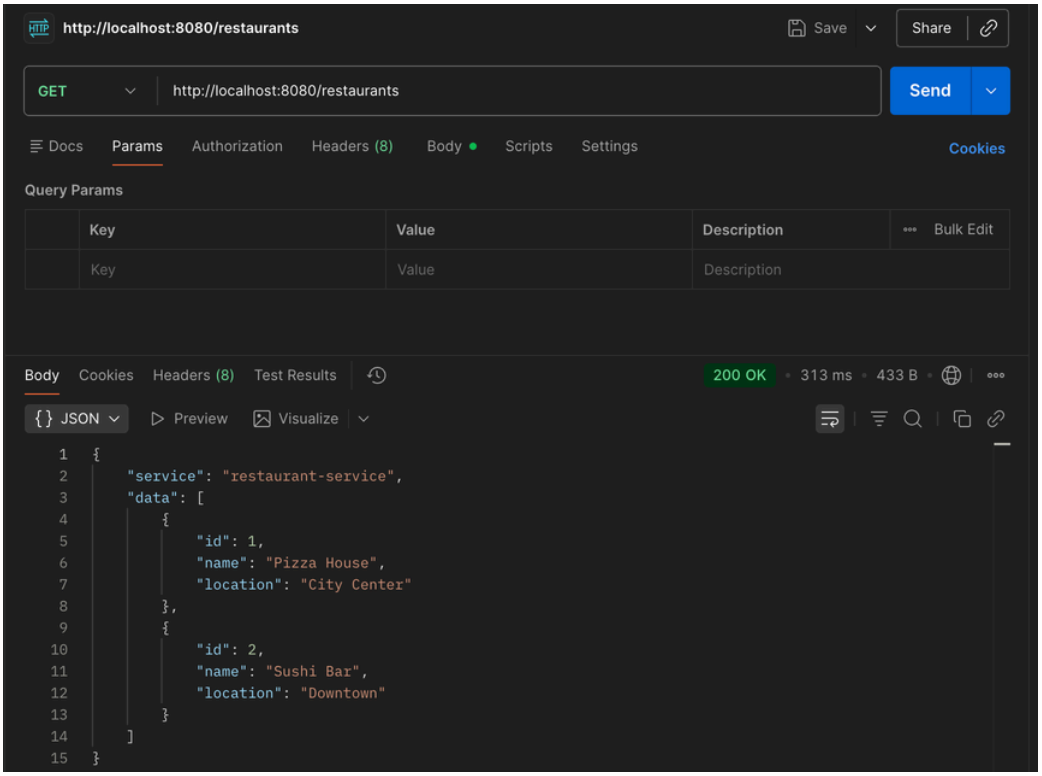


Figure 2 – Successful GET request routed through API Gateway (port 8080)

Inter-Service Communication

In this system, services do not work completely independently. The order-service depends on the menu-service to validate menu items before creating an order.

When a client sends a POST request to /orders, the request first reaches the API Gateway and is forwarded to the order-service. Before saving the order, the order-service makes an internal HTTP request to the menu-service using the endpoint /menus/item/:id. This validation step ensures that the requested item exists and retrieves its price information. If the item is not found, the order is not created and an appropriate error response is returned.

This design avoids duplicating menu data inside the order-service. Each service remains responsible only for its own domain. The menu-service manages menu data, while the order-service focuses on order processing and persistence.

Communication between services happens over HTTP within the isolated Docker network. Service discovery is handled automatically by Docker using service names defined in docker-compose.yml.

This approach keeps the system loosely coupled and easier to maintain.

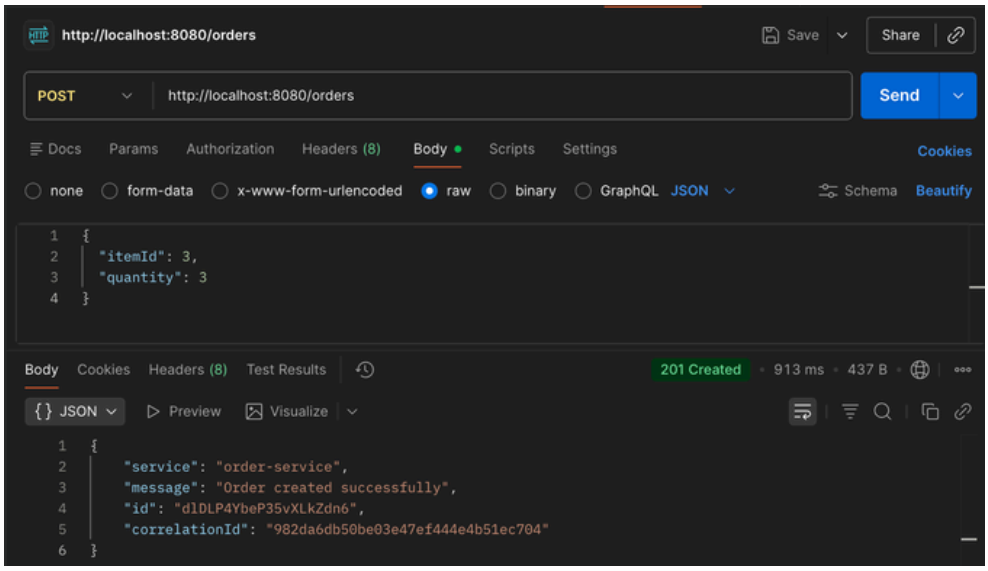


Figure 3 – Order creation with inter-service validation

Cloud Integration and Persistence

The order-service integrates with Google Firebase Firestore to provide persistent storage for customer orders.
Firestore was chosen because it is a managed cloud database that does not require server setup or infrastructure management. It allows the system to store data externally, outside of the Docker containers. This ensures that order data is not lost when containers are restarted.
When an order is successfully validated, the order-service creates a new document inside the orders collection in Firestore. The document contains the item information, quantity, price, status, and timestamp.
By separating the database from the application containers, the system follows cloud-native principles. The services remain stateless, while persistence is handled by a managed cloud service. This design improves reliability and reflects how modern backend systems are built in real production environments.

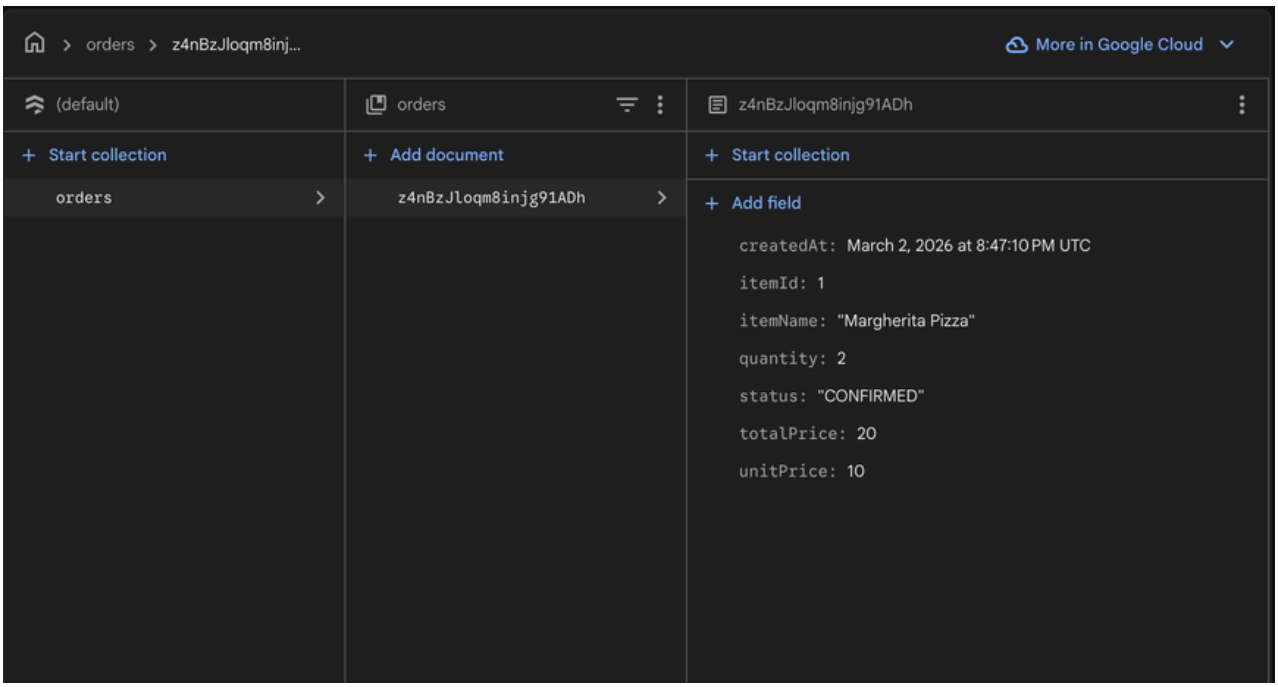


Figure 4 – Order stored in Firebase Firestore

Observability and Correlation ID

To improve traceability across services, the system uses a correlation ID mechanism.
The API Gateway generates an X-Correlation-ID for every incoming request. This ID is automatically forwarded to all internal services through HTTP headers.
Each service logs the correlation ID together with the request method, URL, response status, and execution time. When a request triggers inter-service communication (for example, order-service calling menu-service), the same correlation ID is forwarded to the second service.
This makes it possible to trace a single client request across multiple services by following the same ID in the logs. Even though this is a small system, this mechanism reflects how distributed tracing works in real microservices architectures.

```
Attaching to api-gateway-1, menu-service-1, order-service-1, restaurant-service-1
restaurant-service-1  | Restaurant Service running on port 3001
menu-service-1        | Menu Service running on port 3002
api-gateway-1         | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
api-gateway-1         | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
api-gateway-1         | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
api-gateway-1         | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
api-gateway-1         | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
api-gateway-1         | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
api-gateway-1         | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
api-gateway-1         | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
api-gateway-1         | /docker-entrypoint.sh: Configuration complete; ready for start up
order-service-1       | Order Service running on port 3003
menu-service-1        | [982da6db50be03e47ef444e4b51ec704] GET /menus/item/3 -> 200 (6ms)
order-service-1       | [982da6db50be03e47ef444e4b51ec704] POST /orders -> 201 (859ms)
api-gateway-1         | 192.168.65.1 - - [02/Mar/2026:23:44:24 +0000] "POST /orders HTTP/1.1" 201 145 "-" "PostmanRuntime/7.51.1"
```

Figure 5 – Correlation ID propagation across services

Conclusion

This project demonstrates the design and implementation of a cloud-native backend system using microservices architecture. The system separates responsibilities across independent services and uses an API Gateway to control external access. Docker was used to containerize and isolate services, while Firebase Firestore was integrated as a managed cloud database to ensure persistent storage. Inter-service communication was implemented using REST APIs, and correlation IDs were introduced to improve request tracing across services. Overall, the architecture reflects core cloud-native principles such as service isolation, stateless design, externalized configuration, and managed cloud integration. The system is modular, extensible, and aligned with modern backend design practices.

AI Usage Declaration

During the development of this project, ChatGPT was used as a coding assistant to help generate parts of the implementation code